

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC

Lớp: An toàn và bảo mật dữ liệu trong hệ thống thông tin - CQ2023/1

23120087 - Phạm Ngọc Thanh

23120289 - Nguyễn Nhật Lâm

23120306 - Đào Nguyễn Nguyên

23120312 - Thạch Như

23120411 - Thạch Ý

GIẢNG VIÊN HƯỚNG DẪN

TS. Phạm Thị Bạch Huệ

ThS. Lương Vĩ Minh

ThS. Tiết Gia Hồng

Thành phố Hồ Chí Minh, Tháng 06/2026

Mục lục

Mục lục	2
Danh sách chức năng đã hoàn tất	3
Phân công công việc và Đánh giá	3
Phân hệ 1	3
Phân hệ 2	4
Phân hệ 1: ỨNG DỤNG QUẢN TRỊ CSDL ORACLE	5
Phân hệ 2: ỨNG DỤNG QUẢN LÝ DỮ LIỆU Y TẾ	6
1 Yêu cầu 1: giải pháp cấp quyền truy cập, cài đặt giao diện - RBAC	6
1.1 Bài toán và giải pháp	6
1.2 Thuyết minh kết quả đạt được	6
1.3 Nhận xét, đánh giá và bài học kinh nghiệm	6
2 Yêu cầu 1: giải pháp cấp quyền truy cập, cài đặt giao diện - VPD	6
2.1 Bài toán và giải pháp	6
2.2 Thuyết minh kết quả đạt được	6
2.3 Nhận xét, đánh giá và bài học kinh nghiệm	6
3 Yêu cầu 2: cơ chế phát tán thông báo dùng OLS, cài đặt giao diện	7
3.1 Bài toán và giải pháp	7
3.2 Thuyết minh kết quả đạt được	7
3.3 Nhận xét, đánh giá và bài học kinh nghiệm	7
4 Yêu cầu 3: vận dụng cơ chế kiểm toán	7
4.1 Bài toán và giải pháp	7
4.2 Thuyết minh kết quả đạt được	7
4.3 Nhận xét, đánh giá và bài học kinh nghiệm	8
5 Yêu cầu 4: sao lưu và phục hồi dữ liệu	8
5.1 Bài toán và giải pháp	8
5.2 Thuyết minh kết quả đạt được	8
5.3 Nhận xét, đánh giá và bài học kinh nghiệm	8
Tài liệu tham khảo (theo chuẩn IEEE)	8

Danh sách chức năng đã hoàn tất

Phân công công việc và Đánh giá

Phân hệ 1

Việc tìm hiểu và cài đặt Phân hệ 1 của Đồ án môn học được chia thành 5 công việc sau:

- Task 1. Lập trình giao diện quản lý Role, demo các nghiệp vụ liên quan đến Role và cấp/thu hồi quyền.
- Task 2. Lập trình cơ chế phân quyền (Grant/Revoke), bao gồm việc viết truy vấn SQL xử lý quyền mức hệ thống/mức cột và lập trình Tab Quản lý Quyền hạn tương ứng.
- Task 3. Khởi tạo dữ liệu mẫu (Data Mocking), tổng hợp tài liệu báo cáo kỹ thuật và video demo.
- Task 4. Thiết lập tài khoản DBA, viết SQL cho User/Role, xây dựng giao diện Winform cơ sở kèm Tab tra cứu thông tin quyền, và thực hiện demo luồng nghiệp vụ User.
- Task 5. Phân tích ERD, khởi tạo Schema và lập trình chức năng cho Tab Quản lý User trên ứng dụng.

Bảng phân công và đánh giá

STT	MSSV	Họ và tên	Phân công	Đóng góp	Hoàn thành
1	23120087	Phạm Ngọc Thanh	Task 1	23%	100%
2	23120289	Nguyễn Nhật Tân	Task 2	22%	90%

3	23120306	Đào Nguyễn Nguyên	Task 3	10%	100%
4	23120312	Thạch Như	Task 4	23%	100%
5	23120411	Thạch Ý	Task 5	22%	100%

Phân hệ 2

Việc tìm hiểu và cài đặt Phân hệ 2 của Đồ án môn học được chia thành 5 công việc sau:

- Task 1. Yêu cầu 1: giải pháp cấp quyền truy cập và cài đặt giao diện: cài đặt phân quyền tĩnh (RBAC) và xây dựng UI cho “Kỹ thuật viên” và “Bệnh nhân”.
- Task 2. Yêu cầu 1: giải pháp cấp quyền truy cập và cài đặt giao diện: cài đặt bảo mật mức dòng (VPD) và xây dựng UI cho “Điều phối viên” và “Y sĩ/Bác sĩ”.
- Task 3. Yêu cầu 2: cơ chế phát tán thông báo dừng OLS và cài đặt giao diện.
- Task 4. Yêu cầu 3, yêu cầu 4: viết script thiết lập kiểm toán (FGA) và lập trình các Procedure Backup và Restore (Data pump, flashback).
- Task 5. Yêu cầu 3, yêu cầu 4: viết script cho kiểm toán (Standard Auditing), RMAN Backup và Restore, làm UI quản trị (xem Log, thao tác Backup/Restore), test tích hợp toàn hệ thống, hướng dẫn Build/Run và demo.

Bảng phân công và đánh giá

STT	MSSV	Họ và tên	Phân công	Đóng góp	Hoàn thành
1	23120087	Phạm Ngọc Thanh	Task 3	20%	100%
2	23120289	Nguyễn Nhật Lân	Task 2	20%	100%
3	23120306	Đào Nguyễn Nguyên	Task 4	20%	100%
4	23120312	Thạch Như	Task 5	20%	100%
5	23120411	Thạch Ý	Task 1	20%	100%

Nhóm trưởng: Đào Nguyễn Nguyên

Phân hệ 1: ỨNG DỤNG QUẢN TRỊ CSDL ORACLE

Phân hệ 2: ỨNG DỤNG QUẢN LÝ DỮ LIỆU Y TẾ

1 Yêu cầu 1: giải pháp cấp quyền truy cập, cài đặt giao diện - RBAC

1.1 Bài toán và giải pháp

(chỉ là câu trúc đề xuất tham khảo thôi nha)

Bài toán:

Cơ sở lý thuyết:

Giải pháp:

1.2 Thuyết minh kết quả đạt được

1.3 Nhận xét, đánh giá và bài học kinh nghiệm

2 Yêu cầu 1: giải pháp cấp quyền truy cập, cài đặt giao diện - VPD

2.1. Bài toán và giải pháp

2.1.1. Bài toán

Hệ thống quản lý y tế của bệnh viện X yêu cầu kiểm soát truy cập dữ liệu ở mức hàng (row-level) đối với hai vai trò có đặc quyền cao là Điều phối viên và Bác sĩ/Y sĩ. Cụ thể:

- Điều phối viên (TC#2): Có thể xem, thêm, sửa dữ liệu BỆNH NHÂN; tạo mới HSBA; điều phối bác sĩ (cập nhật MAKHOA, MABS) và điều phối kỹ thuật viên (MAKTV) trong HSBA_DV. Điều phối viên cần thấy toàn bộ dữ liệu HSBA và BỆNH NHÂN để thực hiện chức năng điều phối.
- Bác sĩ/Y sĩ (TC#3): Chỉ được xem và thao tác trên các hồ sơ bệnh án (HSBA) mà chính bác sĩ đó phụ trách (MABS = tài khoản đang đăng nhập). Theo đó, bác sĩ chỉ thấy BỆNH NHÂN, HSBA_DV và ĐƠN THUỐC liên quan đến HSBA của mình.

Yêu cầu này không thể giải quyết đơn thuần bằng RBAC (Role-Based Access Control) vì RBAC chỉ kiểm soát ở mức đối tượng (bảng/view), không phân biệt được từng dòng dữ liệu thuộc về ai. Do đó, cần áp dụng cơ chế VPD (Virtual Private Database).

2.1.2. Cơ sở lý thuyết

VPD (Virtual Private Database), hay còn gọi là Fine-Grained Access Control (FGAC) hoặc Row-Level Security (RLS), là tính năng của Oracle cho phép tự động gắn thêm điều kiện WHERE vào mọi câu truy vấn SQL trên một bảng hoặc view được bảo vệ.

Cơ chế hoạt động:

- Người quản trị định nghĩa một Policy Function – hàm PL/SQL nhận vào tên schema và tên bảng, trả về một chuỗi điều kiện WHERE (ví dụ: MABS = 'BS001').
- Hàm này được đăng ký vào một Policy thông qua thủ tục DBMS_RLS.ADD_POLICY, gắn với bảng mục tiêu và loại thao tác (SELECT, INSERT, UPDATE, DELETE).
- Mỗi khi người dùng thực thi câu lệnh SQL trên bảng đó, Oracle Engine tự động gọi hàm Policy Function để lấy điều kiện WHERE tương ứng với người dùng hiện tại (thông qua SYS_CONTEXT('USERENV', 'SESSION_USER')), rồi nối thêm vào câu SQL gốc.
- Điều kiện này hoàn toàn trong suốt với người dùng – họ không thể biết hay bỏ qua nó.

Ưu điểm của VPD so với RBAC:

Tiêu chí	RBAC	VPD
Mức kiểm soát	Mức đối tượng (bảng, cột)	Mức hàng dữ liệu
Tính linh hoạt	Tĩnh – quyền cố định theo role	Động – điều kiện thay đổi theo ngữ cảnh người dùng
Độ trong suốt	Người dùng biết quyền của mình	Hoàn toàn trong suốt với người dùng
Phù hợp với bài toán	TC#4 (KTV), TC#5 (BN) – phân quyền đơn giản	TC#2 (ĐPV), TC#3 (BS) – phân quyền theo dữ liệu

2.1.3. Giải pháp

Nhóm áp dụng VPD cho 4 bảng chính: HSBA, BENHNNHAN, HSBA_DV, DONTHUOC, với mỗi bảng có 4 policy riêng biệt tương ứng với 4 loại thao tác SELECT, INSERT, UPDATE, DELETE. Tất cả các policy trên một bảng dùng chung một hàm Policy Function, giúp đơn giản hóa việc bảo trì.

Điều kiện lọc dữ liệu được xác định theo vai trò của người dùng đang đăng nhập, được tra cứu động từ bảng NHANVIEN thông qua SYS_CONTEXT('USERENV', 'SESSION_USER').

Bảng tổng hợp các Policy đã cài đặt:

Bảng	Tên Policy	Thao tác	Điều kiện – Điều phối viên	Điều kiện – Bác sĩ/Y sĩ
HSBA	POL_HSBA_SEL	SELECT	1=1 (toàn bộ)	MABS = SESSION_USER
HSBA	POL_HSBA_INS	INSERT	1=1	1=2 (không được phép)
HSBA	POL_HSBA_UPD	UPDATE	1=1	MABS = SESSION_USER
HSBA	POL_HSBA_DEL	DELETE	1=2 (không được phép)	1=2 (không được phép)

BENHNHAN	POL_BENHNHAN_SEL	SELECT	1=1 (toàn bộ)	MABN IN (SELECT MABN FROM HSBA WHERE MABS = SESSION_USER)
BENHNHAN	POL_BENHNHAN_UPD	UPDATE	1=1	MABN IN (SELECT MABN FROM HSBA WHERE MABS = SESSION_USER)
HSBA_DV	POL_HSBADV_SEL	SELECT	MAHSBA IN (SELECT MAHSBA FROM HSBA WHERE MABS IS NOT NULL)	MAHSBA IN (SELECT MAHSBA FROM HSBA WHERE MABS = SESSION_USER)
HSBA_DV	POL_HSBADV_INS/DEL	INSERT, DELETE	1=2 (không được phép)	MAHSBA IN (...MABS = SESSION_USER)
DONTHUOC	POL_DONTHUOC_*	SELECT/INS/UPD/DEL	1=2 (không truy cập)	MAHSBA IN (...MABS = SESSION_USER)

2.2. Thuyết minh kết quả đạt được

2.2.1. Cài đặt Policy Function và đăng ký Policy

Bước 1 – Cấp quyền (GRANT) cho các Role trước khi áp dụng VPD. VPD kiểm soát ở mức hàng, nhưng vẫn cần RBAC cấp quyền ở mức bảng/cột trước:

Bước 2 – Tạo Policy Function

Bước 3 – Đăng ký Policy

2.2.2. Kiểm chứng kết quả

Sau khi đăng ký các policy, nhóm tiến hành kiểm chứng bằng cách đăng nhập với tài khoản của từng vai trò:

Kịch bản 1 – Bác sĩ BS001 đăng nhập và truy vấn HSBA:

```
-- Đăng nhập bằng tài khoản BS001
SELECT * FROM ADMIN_PHANHE1.HSBA;

-- Oracle tự động chuyển thành:
-- SELECT * FROM ADMIN_PHANHE1.HSBA WHERE MABS = 'BS001'
```

Kết quả: Chỉ hiển thị các hồ sơ có MABS = 'BS001'

Kịch bản 2 – Điều phối viên DPV001 truy vấn HSBA:

```
-- Đăng nhập bằng tài khoản DPV001
SELECT * FROM ADMIN_PHANHE1.HSBA;

-- Oracle tự động chuyển thành:
-- SELECT * FROM ADMIN_PHANHE1.HSBA WHERE 1=1
```

Kết quả: Hiển thị toàn bộ hồ sơ bệnh án

Kịch bản 3 – Bác sĩ BS001 xem danh sách bệnh nhân:

```
SELECT * FROM ADMIN_PHANHE1.BENHNHAN;

-- Oracle chuyển thành:
-- SELECT * FROM ADMIN_PHANHE1.BENHNHAN
-- WHERE MABN IN (
```

```
--      SELECT MABN FROM ADMIN_PHANHE1.HSBA
--      WHERE MABS = 'BS001'
--  )
```

Kết quả: Chỉ thấy bệnh nhân có hồ sơ do BS001 điều trị

2.2.3. Giao diện cài đặt

Nhóm xây dựng giao diện WinForm tương ứng cho từng vai trò. Khi đăng nhập, hệ thống tự động xác định vai trò và hiển thị giao diện phù hợp. Dữ liệu hiển thị trên giao diện đã được VPD lọc sẵn ở tầng CSDL, không cần thêm điều kiện lọc ở tầng ứng dụng.

Màn hình	Điều phối viên	Bác sĩ/Y sĩ
Danh sách HSBA	Hiện toàn bộ HSBA, có thể tạo mới, phân công bác sĩ và KTV	Chỉ hiện HSBA của bác sĩ đó; có thể cập nhật CHANDOAN, DIEUTRI, KETLUAN
Danh sách bệnh nhân	Xem, thêm, sửa toàn bộ BỆNNHÂN	Chỉ thấy BN liên quan HSBA mình phụ trách; sửa được TIENSUBENH, DIUNGTHUOC
Dịch vụ (HSBA_DV)	Xem và phân công KTV (cập nhật MAKTV)	Thêm, xóa dịch vụ trong HSBA của mình
Đơn thuốc	Không có quyền truy cập	Thêm, xóa, sửa ĐƠNTHUỐC liên quan HSBA của mình

2.3. Nhận xét, đánh giá và bài học kinh nghiệm

2.3.1. Ưu điểm của giải pháp VPD đã áp dụng

- Bảo mật tầng CSDL triệt để: Điều kiện lọc được áp dụng trực tiếp trong Oracle Engine, không phụ thuộc vào tầng ứng dụng. Ngay cả khi người dùng kết nối trực tiếp qua SQL*Plus hay công cụ khác, dữ liệu vẫn được lọc đúng.
- Trong suốt với người dùng và ứng dụng: Câu lệnh SQL ở tầng ứng dụng không cần thay đổi. Toàn bộ logic phân quyền nằm trong hàm Policy Function, giúp tách biệt rõ ràng giữa logic nghiệp vụ và logic bảo mật.
- Dễ mở rộng và bảo trì: Khi cần thay đổi chính sách bảo mật (ví dụ bổ sung vai trò mới), chỉ cần cập nhật hàm Policy Function mà không cần sửa đổi ứng dụng.
- Dùng chung một hàm cho nhiều loại thao tác: Mỗi bảng chỉ cần một hàm VPD nhưng đăng ký thành 4 policy riêng cho SELECT/INSERT/UPDATE/DELETE, giảm thiểu code trùng lặp.
- update_check = TRUE cho INSERT: Ngăn bác sĩ insert dữ liệu không thuộc phạm vi được phép (ví dụ thêm HSBA với MABS của người khác), tăng cường tính toàn vẹn dữ liệu.

2.3.2. Hạn chế và lưu ý

- Ảnh hưởng hiệu năng: Mỗi truy vấn đều phải gọi hàm Policy Function, bao gồm một lần SELECT vào bảng NHANVIEN để tra cứu vai trò. Với hệ thống có lượng truy vấn lớn, cần cân nhắc dùng Application Context (DBMS_SESSION) để cache thông tin vai trò, tránh truy vấn lặp lại.
- Phụ thuộc dữ liệu: Policy Function tra cứu vai trò từ bảng NHANVIEN. Nếu bảng này có vấn đề hoặc tài khoản không tồn tại trong bảng (ví dụ tài khoản bệnh nhân), hàm sẽ trả về '1=2' và chặn hoàn toàn truy cập – đây là hành vi an toàn nhưng cần lưu ý.
- Khó debug: Khi người dùng phàn nàn không thấy dữ liệu, cần phân biệt giữa dữ liệu thực sự rỗng hay bị VPD lọc. Nên có công cụ kiểm tra policy đang hoạt động (truy vấn DBA_POLICIES).

2.3.3. Bài học kinh nghiệm

1. Cần phân biệt rõ VPD và RBAC: VPD kiểm soát dữ liệu thấy được (dòng nào), RBAC kiểm soát hành động được phép (thao tác nào trên cột nào). Hai cơ chế bổ sung cho nhau và cần được phối hợp đúng cách.

2. Thứ tự triển khai quan trọng: Phải GRANT quyền trước khi ADD_POLICY. Nếu role chưa có quyền SELECT, dù VPD trả về '1=1' người dùng vẫn không xem được dữ liệu.
3. Tham số update_check trong INSERT policy: Đây là tham số quan trọng thường bị bỏ qua. Nếu không có update_check = TRUE, người dùng có thể INSERT dữ liệu ngoài phạm vi được phép nhưng không đọc lại được.
4. Xử lý ngoại lệ trong Policy Function: Luôn có khối EXCEPTION để xử lý trường hợp NO_DATA_FOUND (người dùng không tồn tại trong NHANVIEN), trả về '1=2' để chặn truy cập thay vì gây lỗi.
5. Kiểm chứng kỹ từng kịch bản: Cần kiểm tra cả trường hợp hợp lệ (bác sĩ xem HSBA của mình) và không hợp lệ (bác sĩ cố truy cập HSBA của người khác) để đảm bảo policy hoạt động đúng.

3. Yêu cầu 2: Cơ chế phát tán thông báo bằng Oracle Label Security (OLS) và cài đặt giao diện

3.1. Bài toán và Giải pháp

a. Bài toán

Bệnh viện có nhu cầu phân phối thông báo theo một mô hình phân tán và phân cấp phức tạp. Yêu cầu cốt lõi là đảm bảo nguyên tắc đặc quyền tối thiểu: mỗi cá nhân chỉ được tiếp cận các thông báo phù hợp với chức vụ, chuyên môn và khu vực công tác. Đối tượng nhận thông báo có thể đa dạng:

- **Toàn viện hoặc Từng cấp bậc** (Ban giám đốc, Lãnh đạo khoa, Nhân viên).
- **Từng chuyên khoa** (Tiêu hóa, Thần kinh, Tim mạch).
- **Từng cơ sở** (Cơ sở chung, TP. Hồ Chí Minh, Hà Nội, Hải Phòng).

Nếu áp dụng phương pháp kiểm soát dữ liệu (bảo mật cấp dòng) ở tầng ứng dụng (Application) thông qua các mệnh đề WHERE, hệ thống sẽ gặp các vấn đề:

- Câu truy vấn phức tạp, cồng kềnh và khó bảo trì.
- Rủi ro bảo mật cao (Bypass): Tin tặc hoặc người dùng nội bộ có thể vượt qua màng lọc này nếu truy cập trực tiếp vào hệ quản trị cơ sở dữ liệu (CSDL).

b. Cơ sở lý thuyết

Để giải quyết triệt để vấn đề, giải pháp được lựa chọn là **Oracle Label Security (OLS)**. Đây là kiến trúc bảo mật cấp cao của Oracle dựa trên cơ chế Kiểm soát truy cập dựa trên Nhãn (Label-based Access Control). Mỗi dòng dữ liệu và mỗi người dùng đều được gán một "Nhãn" cấu thành từ 3 thành phần:

- **Levels (Bắt buộc):** Thể hiện sự phân cấp dọc. Người có cấp độ cao có thể đọc dữ liệu ở cấp độ của mình và các cấp thấp hơn.
- **Compartments (Tùy chọn):** Phân chia theo khu vực phi thứ bậc. Người dùng phải sở hữu *tất cả* các khoang mà dữ liệu yêu cầu mới được phép truy cập.
- **Groups (Tùy chọn):** Phân chia theo cấu trúc cây. Nút cha có quyền truy cập dữ liệu của các nút con trực thuộc.

c. Giải pháp triển khai chi tiết

Hệ thống triển khai OLS lên bảng THONGBAO với chính sách bảo mật mang tên OLS_BV. Cấu trúc nhân được thiết kế chuẩn hóa như sau:

Thành phần	Ký hiệu	Giá trị	Ý nghĩa
Levels (Phân cấp)	GD	30	Ban Giám đốc (Cao nhất)
	LD	20	Lãnh đạo
	NV	10	Nhân viên (Thấp nhất)
Compartments (Chuyên khoa)	TH	100	Khoa Tiêu hóa
	TK	110	Khoa Thần kinh
	TM	120	Khoa Tim mạch
Groups (Chi nhánh)	TOAN_VIE N	100	Cấp toàn viện (Root)
	HCM	10	Chi nhánh TP. HCM
	HN	20	Chi nhánh Hà Nội

	HP	30	Chi nhánh Hải Phòng
--	----	----	---------------------

d. Luồng thực thi hệ thống

- Khởi tạo Policy OLS_BV, thêm cột ẩn OLS_LABEL vào bảng ADMIN_PHANHE1.THONGBAO.
- Tạo danh sách các chuỗi nhãn hợp lệ (VD: LD:TH, NV:TH:HCM).
- Tự động gán nhãn cho dữ liệu THONGBAO tại thời điểm INSERT.
- Cấp phát quyền OLS (Label) cho các tài khoản kiểm thử (U1 đến U8) theo đúng vai trò thực tế.
- Kích hoạt Policy trên bảng với các tùy chọn kiểm soát nghiêm ngặt: READ_CONTROL, WRITE_CONTROL, CHECK_CONTROL.
- Trên ứng dụng C#, cấu hình Connection String kết nối trực tiếp vào Pluggable Database (xepdb1).

3.2. Thuyết minh kết quả đạt được

Hệ thống đã tích hợp thành công C# và cơ chế OLS dưới tầng CSDL.

- **Truy vấn tối giản:** Tại Form *Xem thông báo*, ứng dụng C# chỉ cần thực thi một câu lệnh duy nhất: SELECT * FROM ADMIN_PHANHE1.THONGBAO. Hoàn toàn không có mệnh đề WHERE để lọc dữ liệu.
- **Lọc dữ liệu tự động (Cấp CSDL):** Kết quả trả về cho các tài khoản đăng nhập (U1 - U8) được Oracle tự động sàng lọc chính xác theo kịch bản phân quyền:
 - **Tài khoản U1 (Giám đốc toàn viện):** Truy xuất được 100% thông báo do sở hữu Level cao nhất (GD), Group cha (TOAN_VIEN) và toàn quyền trên các Compartment.
 - **Tài khoản U2 (Lãnh đạo Tim mạch - HCM):** Chỉ xem được thông báo cấp Toàn viện (dành cho Lãnh đạo/Nhân viên) và thông báo nội bộ của khoa Tim mạch cơ sở HCM. Các thông báo thuộc khoa khác hoặc cơ sở khác bị ẩn hoàn toàn.
 - **Tài khoản U8 (Nhân viên Tiêu hóa - Hà Nội):** Chỉ xem được thông báo nội bộ Khoa Tiêu hóa tại Hà Nội và các thông báo chung cấp thấp (NV).

Kết luận: Dữ liệu hiển thị trên DataGrid của UI hoàn toàn trùng khớp với ma trận bảo mật ban đầu, khẳng định tính đúng đắn của giải pháp.

3.3. Đánh giá và bài học kinh nghiệm

a. Nhận xét và đánh giá

- **Ưu điểm:**
 - **Bảo mật hướng dữ liệu:** Khắc phục triệt để lỗi hồng truy cập trái phép. Mọi nỗ lực truy xuất, dù qua C# hay đăng nhập trực tiếp (qua DataGrip/SQL Developer), đều bị từ chối nếu không đủ thẩm quyền Label.

- **Tối ưu hóa mã nguồn:** Chuyển giao toàn bộ logic phân quyền phức tạp xuống CSDL, giúp mã nguồn C# ở tầng Application trở nên gọn nhẹ, sạch sẽ và dễ bảo trì.
- **Hạn chế:** Đòi hỏi kiến thức chuyên sâu về quản trị kiến trúc Pluggable Database (CDB/PDB). Quá trình hoạch định ma trận phân quyền ban đầu tiêu tốn nhiều thời gian và công sức để đảm bảo không xảy ra xung đột logic.

b. Bài học kinh nghiệm

- **Thiết kế kỹ lưỡng trước khi lập trình:** Cần lập bảng ma trận phân quyền (Matrix) rõ ràng trên giấy cho các thành phần (Level, Compartment, Group). Một sai sót nhỏ trong thiết kế nhãn có thể dẫn đến việc dữ liệu bị ẩn hoàn toàn gây lỗi logic trên diện rộng.
- **Quản lý Session Connection:** Do OLS được cấu hình ở mức Pluggable Database, Connection String trong C# bắt buộc phải trỏ chính xác đến SERVICE_NAME=xepdb1 (hoặc tên PDB tương ứng), tuyệt đối không dùng Root Container.

4 Yêu cầu 3: vận dụng cơ chế kiểm toán

4.1 Bài toán và giải pháp

Bài toán:

Hệ thống quản lý dữ liệu y tế chứa các thông tin nhạy cảm nên cần được giám sát chặt chẽ nhằm phát hiện các hành vi truy cập và chỉnh sửa dữ liệu bất thường. Yêu cầu đặt ra là phải thiết lập kiểm toán hệ thống theo 2 mức độ:

- Giám sát cơ bản các hành vi truy cập đối tượng (Standard Auditing).
- Giám sát hạt mịn (Fine-Grained Auditing) hoặc là unified auditing dựa trên ngữ cảnh dữ liệu và nghiệp vụ.

Cơ sở lý thuyết:

Kiểm toán (Audit) là cơ chế ghi lại mọi hoạt động trên database. Nó giống như một máy quay trong nhà, quay lại mọi hoạt động và trả lời các câu hỏi: Ai đã làm?, Làm cái gì?, Làm lúc nào?, Làm như thế nào? và Có thực hiện thành công không?. Kiểm toán thường được dùng để truy vết đối với những hành vi, những người dùng có thẩm quyền cao, truy cập đến những dữ liệu nhạy cảm, có giá trị. [1]

a. Standard auditing:

b. Unified auditing:

Là một cơ chế kiểm toán như một kiến trúc tập trung, gom tất cả nhật ký kiểm toán (từ RMAN, Data Pump, Database Vault...) vào một nơi duy nhất. Do đó, unified audit mạnh về việc giám sát hành động ở mức tổng thể. [1]

c. Fine-grained auditing:

FGA cung cấp khả năng kiểm toán chi tiết hơn, dựa trên nội dung dữ liệu (Value-based auditing). Thông qua package DBMS_FGA, FGA cho phép chỉ định chính xác cột (audit_column), điều kiện (audit_condition) kết hợp với các hàm PL/SQL phức tạp và giám sát dựa trên giá trị trả về của từng cột, điều mà unified auditing không làm được. [1]

Giải pháp:

a. Standard Auditing:

Thiết lập tham số audit_trail = DB, EXTENDED để bắt được câu lệnh SQL. Cài đặt 5 ngữ cảnh:

- Giám sát đăng nhập thất bại
- DML trên bảng cột lỗi (HSBA)
- Select trên view xem audit log (V_ALL_AUDIT_LOG)
- Thực thi thủ tục nhảy cảm (SP_RESTORE_FLASHBACK)
- DDL thay đổi cấu trúc Database.

b. Fine-Grained Auditing:

Để thực hiện các điều kiện kiểm toán phức tạp của mỗi tình huống, nhóm đề xuất giải pháp sử dụng các hàm trả về kiểu VARCHAR có giá trị là TRUE hoặc FALSE để truyền vào làm điều kiện thay vì viết trực tiếp các điều kiện phức tạp vào audit_condition. Để vượt qua cơ chế sandbox vô hiệu hóa role của Oracle PDB (user thực hiện hành động -> hệ thống so sánh với chính sách -> chạy hàm để kiểm tra điều kiện), thay vì để PL/SQL tự kiểm tra quyền bằng SYS_CONTEXT, ta lấy ngữ cảnh trực tiếp từ SQL Engine ở mệnh đề audit_condition và truyền nó vào Function như một tham số hằng số. Điều này

đảm bảo FGA luôn nhận diện đúng quyền của người dùng mà không bị đánh lừa bởi môi trường PDB.

c. Trích xuất log: Tạo một view tổng hợp V_ALL_AUDIT_LOG gộp dữ liệu từ cả 2 hệ thống kiểm toán (DBA_AUDIT_TRAIL cho standard và DBA_FGA_AUDIT_TRAIL cho FGA) để đáp ứng yêu cầu trích xuất dữ liệu.

4.2 Thuyết minh kết quả đạt được

a. Standard auditing:

b. Fine-grained auditing:

Nhóm đã triển khai 7 hàm kiểm tra điều kiện và 8 chính sách để giải quyết 4 yêu cầu:

- Tình huống a: bác sĩ cập nhật hợp pháp các trường nhạy cảm trên DONTUOC

Đã triển khai policy FGA_DONTUOC_CAPNHAT_SAUDINH ghi lại hành vi UPDATE trên các cột MAHSBA, NGAYDT, TENTUOC, LIEUDUNG. Nhóm đã kiểm thử và chính sách hoạt động tốt khi để bác sĩ (NV004) UPDATE (LIEUDUNG) của 1 đơn thuốc được bác sĩ này kê (MAHSBA trên đơn thuốc trùng với MAHSBA của 1 hồ sơ bệnh án mà bác sĩ này điều trị) thì hành vi đã được ghi log.

- Tình huống b: bác sĩ cập nhật hợp pháp các trường nhạy cảm trên HSBA

Đã triển khai policy FGA_HSBA_CAPNHAT_HOPPHAP ghi lại hành vi UPDATE trên các cột CHANDOAN, DIEUTRI, KETLUAN. Nhóm đã kiểm thử và chính sách hoạt động tốt khi để bác sĩ (NV004) UPDATE (KETLUAN) 1 hồ sơ bệnh án có MABS là NV004 thì hành vi đã được ghi log.

- Tình huống c: mọi hành vi cập nhật bất hợp pháp trên HSBA

Đã triển khai policy FGA_HSBA_CAPNHAT_BATHOPPHAP ghi lại hành vi UPDATE bất hợp pháp các cột CHANDOAN, DIEUTRI, KETLUAN. Nhóm đã kiểm thử và chính sách hoạt động tốt khi để ADMIN UPDATE (KETLUAN) 1 hồ sơ bệnh án thì hành vi đã được ghi log.

- Tình huống d: hành vi thêm, xóa, sửa bất hợp pháp trên bảng HSBA_DV

Đã triển khai 4 policy để truy vết thao tác thêm, xóa, sửa trái phép:

- FGA_HSBADV_INS_DEL_BATHOPPHAP: ghi log hành vi INSERT/DELETE nếu không phải bác sĩ.
- FGA_HSBADV_UPD_COT_CAM_BATHOPPHAP: ghi log hành vi UPDATE trên cột vùng cấm (MAHSBA, LOAIDV).

- FGA_HSBADV_UPD_MAKTV_BATHOPPHAP: ghi log hành vi UPDATE cột MAKTV nếu không phải điều phối viên.
 - FGA_HSBADV_UPD_KETQUA_BATHOPPHAP: ghi log hành vi UPDATE cột KETQUA nếu không phải kỹ thuật viên phụ trách.
- Tình huống bổ sung: hành vi kỹ thuật viên cập nhật hợp pháp HSBA_DV (KETQUA).
 Đã triển khai policy FGA_HSBADV_KTV_CAPNHAT_HOPPHAP ghi lại hành vi UPDATE (KETQUA) trên 1 dịch vụ hỗ trợ của kỹ thuật viên phụ trách. Nhóm đã kiểm thử và chính sách hoạt động tốt khi để kỹ thuật viên (NV007) UPDATE (KETQUA) 1 hồ dịch vụ có MAKTV là NV007 thì hành vi đã được ghi log.

4.3 Nhận xét, đánh giá và bài học kinh nghiệm

a. Standard auditing:

b. Fine-grained auditing:

Nhận xét và đánh giá

- Qua quá trình triển khai, nhóm nhận thấy Fine-Grained Audit (FGA) là cơ chế kiểm toán phù hợp đối với các yêu cầu theo dõi hành vi ở mức chi tiết. FGA cho phép kiểm toán dựa trên điều kiện nghiệp vụ cụ thể, giúp phân biệt các hành vi hợp pháp và bất hợp pháp thay vì chỉ ghi nhận thao tác trên đối tượng dữ liệu. Bên cạnh đó, khả năng kết hợp với các hàm PL/SQL giúp đáp ứng các yêu cầu kiểm toán phức tạp liên quan đến nhiều bảng dữ liệu. Tuy nhiên, việc triển khai FGA đòi hỏi phải xử lý cẩn thận các vấn đề về ngữ cảnh thực thi, điều kiện kiểm toán và sự tương tác với các cơ chế bảo mật khác như VPD để đảm bảo kết quả kiểm toán chính xác và đầy đủ.

- Khi kiểm thử với 1 số truy vấn vi phạm chính sách VPD, hệ thống từ chối truy cập ngay từ đầu nên nhóm đã sử dụng các tài khoản có đặc quyền quản trị hoặc tạm thời điều chỉnh phạm vi áp dụng của VPD nhằm tạo ra các tình huống kiểm toán cần thiết.

Bài học kinh nghiệm

Trong quá trình xây dựng và kiểm thử hệ thống FGA, nhóm đã rút ra một số kinh nghiệm quan trọng như sau:

- Xử lý ngữ cảnh bảo mật trong môi trường PDB

Khi triển khai trên môi trường Oracle 21c theo mô hình Pluggable Database (PDB), nhóm nhận thấy một số hàm được FGA gọi gián tiếp không thể sử dụng thông tin vai trò người dùng thông qua SYS_CONTEXT(...) như 'SYS_SESSION_ROLES', 'SESSION_USER' theo cách thông thường. Điều này dẫn đến việc các điều kiện kiểm

toán trả về kết quả không như mong đợi. Nhóm đã khắc phục bằng cách truyền các thông tin ngữ cảnh cần thiết từ tầng SQL vào tham số của hàm PL/SQL thay vì dùng trong nội dung của hàm, giúp các chính sách kiểm toán hoạt động ổn định và chính xác hơn.

+ Giới hạn cú pháp của audit_condition

Nhóm nhận thấy tham số audit_condition hoạt động ổn định nhất khi chỉ chứa các biểu thức kiểm tra đơn giản. Khi biểu thức trở nên phức tạp với nhiều phép kết hợp AND, OR hoặc các truy vấn phụ, nguy cơ phát sinh lỗi phân tích cú pháp hoặc sai lệch tăng lên làm cho chính sách không hoạt động.

Từ đó, nhóm rút ra nguyên tắc thiết kế là phân tách độ phức tạp khỏi audit_condition. Các điều kiện đơn giản nên được khai báo trực tiếp để tránh các lỗi ngầm như lỗi trong PDB, trong khi các điều kiện phức tạp nên được đóng gói trong các hàm PL/SQL riêng biệt và chỉ gọi thông qua audit_condition. Cách tiếp cận này giúp mã nguồn dễ bảo trì hơn, giảm thiểu lỗi cú pháp và nâng cao độ tin cậy của hệ thống kiểm toán.

5 Yêu cầu 4: sao lưu và phục hồi dữ liệu

5.1 Bài toán và giải pháp

Bài toán:

- Backup toàn bộ hệ thống, database (Physical Backup)
- Backup một schema (Logical Backup)
- Khôi phục những thay đổi dựa trên vết Audit Log (Point-in-Time Recovery)

Cơ sở lý thuyết:

a. RMAN:

b. Data pump:

Oracle Data Pump là công nghệ lõi của Oracle hỗ trợ việc di chuyển dữ liệu (data) và siêu dữ liệu (metadata) giữa các cơ sở dữ liệu với tốc độ cực cao.

- Kiến trúc và thành phần: Công nghệ này được cấu thành từ 3 thành phần riêng biệt:
 - Command-line clients (expdp và impdp): Tiện ích dòng lệnh dành cho quản trị viên.

- Gói DBMS_DATAPUMP (Data Pump API): Cung cấp các thủ tục PL/SQL để thực thi xuất/nhập dữ liệu lập trình được. Thực chất, expdp và impdp cũng gọi vào API này.
- Gói DBMS_METADATA (Metadata API): Trích xuất và thao tác với các định nghĩa DDL (Data Definition Language) của từ điển dữ liệu. Trong quá trình thực thi, Data Pump sử dụng một Bảng kiểm soát tác vụ (Control Job Table) để theo dõi tiến độ, kết hợp với các tiến trình con (Worker processes) hoạt động song song để tối đa hóa hiệu suất.

- Các chế độ xuất/nhập (Export/Import Modes): khác với sao lưu vật lý (RMAN) sao lưu toàn bộ block dữ liệu, Data Pump cho phép sao lưu logic với độ linh hoạt rất cao qua 5 chế độ:

- Full Database Mode: Xuất toàn bộ cơ sở dữ liệu (yêu cầu quyền DATAPUMP_EXP_FULL_DATABASE).
- Schema Mode: Xuất một hoặc nhiều Schema cụ thể (bao gồm toàn bộ bảng, view, thủ tục, quyền... thuộc Schema đó).
- Table Mode: Chỉ xuất một bộ các bảng (Tables) hoặc phân vùng (Partitions) được chỉ định.
- Tablespace Mode: Xuất tất cả các bảng nằm trong một hoặc nhiều Tablespace.
- Transportable Tablespace Mode: Di chuyển siêu dữ liệu của Tablespace để sao chép trực tiếp file vật lý sang một CSDL khác, cho tốc độ phục hồi nhanh nhất. [2]

- Đặc thù Server-based (Xử lý tại Máy chủ): Mọi quá trình đọc và ghi file Dump (.dmp) của Data Pump đều được thực thi tại máy chủ Oracle (Server-based), không phải tại máy khách (Client). Do đó, hệ thống bắt buộc phải sử dụng các đối tượng Thư mục (DIRECTORY) được cấu hình trong CSDL để ánh xạ ra đường dẫn vật lý trên hệ điều hành máy chủ, kèm theo sự kiểm soát quyền READ, WRITE nghiêm ngặt.

+ Dùng DBMS_SCHEDULER lập lịch tự động gọi Procedure sao lưu hàng ngày.

c. Flashback:

- Là tập hợp các tính năng cho phép xem lại trạng thái dữ liệu trong quá khứ hoặc "tua ngược" dữ liệu mà không cần phải thực hiện khôi phục từ các tệp tin sao lưu vật lý truyền thống (Point-in-Time Recovery - PITR). Công nghệ này hiệu quả và nhanh hơn rất nhiều so với PITR truyền thống. [3]

Các cấp độ của Flashback bao gồm:

- Physical Flashback (Mức vật lý):

- Flashback Database: Hoạt động ở tầng vật lý, tua ngược toàn bộ cơ sở dữ liệu (hoặc PDB) về một thời điểm trong quá khứ. Nó sử dụng các tệp tin Flashback Logs lưu trong Vùng phục hồi nhanh (Fast Recovery Area). Tính năng này phù hợp để khắc phục lỗi logic diện rộng hoặc khi nâng cấp hệ thống thất bại.

- Logical Flashback (Mức logic):

- Flashback Drop: Sử dụng cơ chế Recycle Bin (Thùng rác) để đảo ngược ngay lập tức lệnh DROP TABLE, khôi phục lại bảng và các đối tượng phụ thuộc.
- Flashback Table: Cho phép khôi phục nội tại một hoặc nhiều bảng về một thời điểm cụ thể trong quá khứ mà không cần đưa cơ sở dữ liệu về trạng thái ngoại tuyến (Offline). Hệ thống tự động xóa các dòng mới và chèn lại các dòng cũ dựa trên Undo Data (Dữ liệu hoàn tác), đồng thời tự động bảo trì các chỉ mục (Indexes) và ràng buộc (Constraints). Điều kiện bắt buộc là bảng phải được bật tính năng di chuyển dòng (ENABLE ROW MOVEMENT).

Giải pháp:

a. RMAN:

b. Data pump:

Mặc dù Data Pump hỗ trợ nhiều cấp độ, nhóm quyết định sử dụng Schema Mode để sao lưu không gian làm việc ADMINBV. Điều này đảm bảo tính đóng gói (Encapsulation) của phân hệ y tế. Thay vì dùng ứng dụng C# gọi lệnh OS (expdp), nhóm sử dụng trực tiếp Gói API DBMS_DATAPUMP để xây dựng các Stored Procedure (SP_BACKUP_DATAPUMP, SP_RESTORE_DATAPUMP).

c. Flashback:

Trong phạm vi khắc phục sự cố sai lệch dữ liệu dựa trên audit log, nhóm lựa chọn công nghệ Flashback Table. Tuy nhiên, vì Flashback Table phụ thuộc vào không gian Undo (vốn có thời gian lưu trữ ngắn và bị ghi đè liên tục), nhóm tích hợp thêm Flashback Data Archive (FDA) để tạo một nơi lưu trữ siêu dữ liệu hoàn tác vĩnh viễn trong 1 năm cho 3 bảng cốt lõi (HSBA, HSBA_DV, DONTHUOC). Xây dựng Stored Procedure SP_RESTORE_FLASHBACK nhận tham số đầu vào là Tên bảng và Mốc thời gian an toàn. Thủ tục này sẽ kết hợp vô hiệu hóa tạm thời các lớp bảo mật (VPD/FGA) để đảm bảo tiến trình Flashback không bị chặn lại.

5.2 Thuyết minh kết quả đạt được

a. RMAN:

b. Data pump:

Sao lưu tự động (Backup): Thủ tục SP_BACKUP_DATAPUMP được cài đặt với bộ lọc SCHEMA_EXPR để chỉ xuất dữ liệu của ADMINBV. Tập dump được tự động sinh tên kèm mốc thời gian chuẩn múi giờ Việt Nam (VD:

BV_PHANHE1_20260413_020000.dmp). Job JOB_DAILY_BACKUP kích hoạt thành công tiến trình này vào lúc 2h00 sáng mỗi ngày. (kiểm thử thực tế là cứ mỗi 5' thì backup 1 lần)

Phục hồi toàn bộ Schema (Restore): Thủ tục SP_RESTORE_DATAPUMP nhận tham số là tên file backup từ C#. Qua thử nghiệm kịch bản giả lập thảm họa (Drop toàn bộ bảng NHANVIEN), tiến trình Import đã chạy ngầm và khôi phục thành công 100% cấu trúc bảng, dữ liệu thô, cũng như các liên kết khóa ngoại.

c. Flashback:

Nhóm đã triển khai thành công cơ chế phục hồi dữ liệu bằng Flashback Table kết hợp Flashback Data Archive (FDA). Hệ thống lưu trữ lịch sử thay đổi dữ liệu của các bảng HSBA, HSBA_DV và DONTUOC trong thời gian 1 năm, cho phép khôi phục dữ liệu về một thời điểm xác định khi xảy ra sự cố. Thủ tục SP_RESTORE_FLASHBACK được xây dựng nhằm tự động hóa toàn bộ quá trình phục hồi, bao gồm kiểm tra đối tượng, tạm thời vô hiệu hóa các chính sách VPD và FGA, thực hiện Flashback Table và kích hoạt lại các cơ chế bảo mật sau khi hoàn tất. Kết quả thử nghiệm cho thấy dữ liệu bị thay đổi trái phép được khôi phục chính xác, nhanh chóng và không làm gián đoạn hoạt động của hệ thống.

5.3 Nhận xét, đánh giá và bài học kinh nghiệm

a. RMAN:

b. Data pump:

Nhận xét và đánh giá:

- Giải pháp sao lưu và phục hồi bằng Oracle Data Pump ở chế độ Schema đáp ứng tốt yêu cầu bảo vệ dữ liệu nghiệp vụ, đồng thời đảm bảo tính linh hoạt trong quá trình vận hành. Việc sử dụng API DBMS_DATAPUMP thay cho các công cụ dòng lệnh expdp/impdp giúp ứng dụng WinForms tích hợp trực tiếp với cơ sở dữ liệu, nâng cao tính bảo mật và khả năng kiểm soát quy trình sao lưu, phục hồi. Tuy nhiên, do Data Pump thực hiện đọc/ghi trên máy chủ nên thời gian sao lưu và phục hồi có thể tăng đáng kể đối với các schema có dung lượng lớn.

Bài học kinh nghiệm:

- Việc lựa chọn API DBMS_DATAPUMP thay vì các công cụ dòng lệnh mang lại nhiều lợi ích về bảo mật và khả năng lập trình. Toàn bộ vòng đời của một Data Pump Job được quản lý trực tiếp trong PL/SQL, giúp kiểm soát lỗi tốt hơn và cung cấp thông tin phản hồi chính xác cho ứng dụng.

- Bên cạnh đó, nhóm nhận thấy tham số TABLE_EXISTS_ACTION cần được lựa chọn phù hợp với từng kịch bản phục hồi. Đối với phục hồi toàn bộ schema, tùy chọn REPLACE giúp Oracle tự động tái tạo đầy đủ cấu trúc bảng và các ràng buộc liên quan. Tuy nhiên, trong trường hợp chỉ phục hồi một số bảng riêng lẻ, việc sử dụng REPLACE có thể làm mất các khóa ngoại tham chiếu từ các bảng khác.

c. Flashback:

Nhận xét và đánh giá

Flashback Table kết hợp Flashback Data Archive là giải pháp hiệu quả để khắc phục các sự cố do thao tác sai của người dùng. Phương pháp này cho phép phục hồi dữ liệu nhanh chóng, chính xác đến một thời điểm xác định và không làm gián đoạn hoạt động của hệ thống, phù hợp với các yêu cầu phục hồi dữ liệu ở mức bảng.

Bài học kinh nghiệm

Quá trình triển khai cho thấy Flashback Table phụ thuộc vào dữ liệu Undo, do đó khả năng phục hồi có thể bị hạn chế khi dữ liệu Undo đã bị ghi đè. Việc sử dụng Flashback Data Archive giúp lưu trữ lịch sử thay đổi dữ liệu trong thời gian dài, nâng cao độ tin cậy của quá trình phục hồi. Ngoài ra, khi áp dụng Flashback cho các bảng đang được bảo vệ bởi VPD và FGA, cần tạm thời vô hiệu hóa các chính sách này trước khi thực hiện phục hồi và kích hoạt lại sau khi hoàn tất để tránh xung đột và đảm bảo hệ thống hoạt động ổn định.

Tài liệu tham khảo (theo chuẩn IEEE)

[1] Oracle Corporation, “Introduction to Auditing,” Oracle Database Security Guide 21c, Oracle Help Center. [Online]. Available: <https://docs.oracle.com/en/database/oracle/oracle-database/21/dbseg/introduction-to-auditing.html>. [Accessed 24 06 2026].

[2] Oracle Corporation, “Oracle Data Pump,” Oracle Database Utilities 21c, Oracle Help Center. [Online]. Available:
<https://docs.oracle.com/en/database/oracle/oracle-database/21/sutil/oracle-data-pump.html>
[Accessed 24 06 2026].

[3] Oracle Corporation, “Performing Flashback Database and Database Point-in-Time Recovery (DBPITR) with RMAN,” Oracle Database Backup and Recovery User’s Guide 21c, Oracle Help Center. [Online]. Available:
<https://docs.oracle.com/en/database/oracle/oracle-database/21/bradv/rman-performing-flashback-dbpitr.html>. [Accessed 24 06 2026].